



**Università
di Genova**

A.A. 2025-26

FisChinG

Un gioco di pesca in grafica 3D

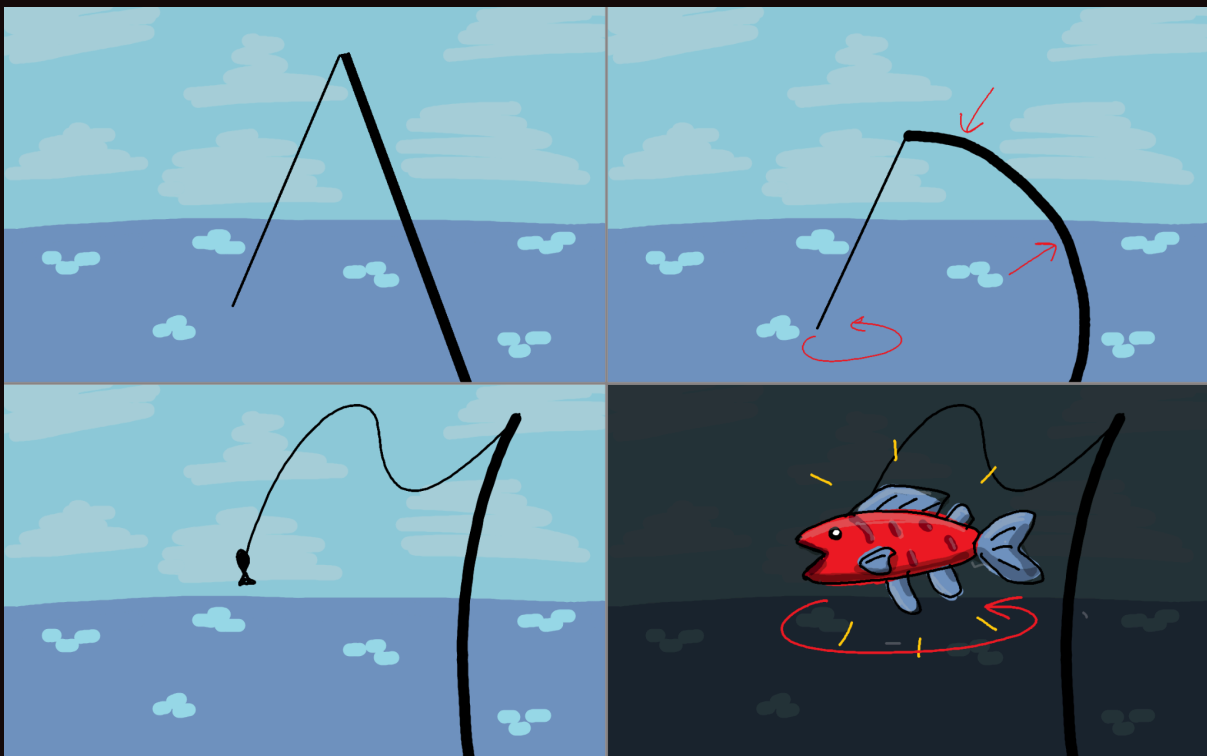
Obiettivo	2
Sviluppo	3
Tappa 1	3
Tappa 2	4
Tappa 3	5
Tappa 4	5
Tappa 5	6
Tappa 6	7
Tappa 7	8
Tappa 8	9
Tappa 9	10
Tappa 10	11
Tappa 11	11
Problemi e Soluzioni	12
Conclusioni	12
Crediti	12

Obiettivo

Il programma da realizzare in questo progetto è un gioco di pesca dalla grafica 3D, implementato quindi utilizzando la libreria OpenGL.

È stato immaginato fin da subito molto semplice, con visuale fissa e in prima persona, gli elementi visibili più importanti sono una canna da pesca, posta di fronte al giocatore, e l'acqua da cui si pesca.

Si è pensato ad un gameplay ciclico, all'inizio è previsto un periodo di attesa, in cui si aspetta che un pesce abbocchi; quando questo succede parte una prova per riuscire a pescarlo, se la cattura ha successo la canna si solleva e a schermo compare il pescce, che ruota su sé stesso per evidenziare il fatto che si trovi in un ambiente tridimensionale, se no si ritorna allo stato iniziale.



Come stile grafico si è pensato di utilizzare modelli low-poly e texture dai pixel visibili, per rendere più semplice la realizzazione delle mesh e dare al gioco uno stile retrò.

Alcuni giochi già esistenti sono stati presi come ispirazione per la direzione artistica:



Sviluppo

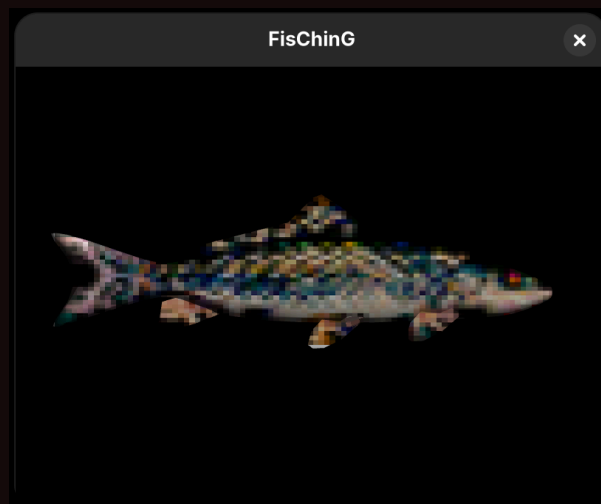
Tappa 1

Per partire sono state prese parti di codice dei laboratori guidati visti a lezione, in particolare le struct Setup e Shaders, la classe Scene e parte del main, in seguito modificati leggermente.

Serviva aggiungere un modo per importare mesh 3D, create esternamente, su OpenGL; è stato quindi implementato un parser di file Wavefront obj, formato utilizzato perché contenente le informazioni strettamente necessarie per questo progetto e disponibile come tipo di esportazione su Blender.

È nata così la classe Model: il costruttore prende in argomento i percorsi relativi di un file .obj, per la mesh, e di un file .png, per la texture, salva in memoria i relativi dati (posizione, coordinate texture e normali), infine passa queste informazioni alla libreria OpenGL, che li elabora rendendoli suoi oggetti disegnabili.

Per testare il codice è stato processato il modello 3D di un pesce.



Tappa 2

È stato effettuato un refactoring del codice.

Gli shader sono stati riscritti in file specifici, .vert e .frag, per elaborarli è stata utilizzata la libreria hot shaders, fornita con i laboratori guidati.

Le classi Model e Setup sono state spostate in file separati.

È stata introdotta la classe ModelCollection per gestire la memorizzazione dei modelli, la classe Scene ne ha un'istanza contenente tutti quelli utilizzati nella scena corrente.

Una nuova struct Entity memorizza, per ogni entità, un puntatore al suo modello, in modo che più oggetti possano usare lo stesso, e posizione e rotazione dell'oggetto stesso, in modo da poter poi calcolare la sua matrice di trasformazione.

Infine è stata scritta una funzione, loadScene che si occupa di istanziare modelli e entità necessari alla costruzione della scena attuale, utile per ridurre la lunghezza del main.

Per testare questa versione del programma sono stati generati due pesci, che usano lo stesso modello ma sono entità separate.



Tappa 3

Sono stati creati dei modelli 3D utilizzando Blender, in particolare una skybox sferica e un panorama che comprende delle colline, un lago e un pontile.

Per poter posizionare correttamente il punto di vista dello spettatore è stata creata la classe Camera.

Per evitare deformazioni dei modelli in seguito ad un ridimensionamento della finestra è stato gestito l'evento corrispondente, che aggiorna la proiezione della camera.



Tappa 4

Per rendere più vivo l'ambiente si è deciso di far muovere delle texture.

Le entità hanno acquisito un campo bool, movement, che stabilisce se la loro texture è da animare o no. Al fragment shader è stata aggiunta una uniform, offset, che indica di quanto va spostata la texture, informazione tenuta aggiornata all'interno di Scene.

Per temporizzare gli spostamenti delle texture è stato istanziato un clock nel main, che permette di ricavare il tempo passato tra un loop principale e l'altro, informazione passata alla classe Scene tramite il metodo update.



Tappa 3

È giunto il momento di aggiungere entità con un comportamento specifico, anziché solo statiche; la prima ad essere implementata è stata la canna da pesca.

Alla classe Entity è stato aggiunto un metodo update dal corpo vuoto, sovrascritto nelle classi derivate per aggiornare i parametri degli oggetti.

È stata quindi aggiunta la classe Rod, derivata di Entity; questa contenente un enum RodState, che definisce i possibili stati della canna da pesca, una serie di parametri necessari per il calcolo dei suoi spostamenti, e la sua personale implementazione di update. Questo metodo, in base allo stato corrente e al tempo passato dall'ultimo aggiornamento, varia la posizione e/o lo stato dell'entità stessa.

In particolare, la canna da pesca ha un comportamento ciclico: all'inizio oscilla poco e lentamente (stato SWAYING), quando un pesce abbocca comincia a vibrare (stato SHAKING), se la pesca ha successo viene sollevata (stati LIFTING e LIFTED) e finita la cattura abbassata nuovamente (stati LOWERING e LOWERED), dopodiché si ricomincia da capo.

Il metodo update della classe Scene è stato aggiornato, è da qui che viene chiamato update su tutte le entità in scena. Inoltre, il metodo chiamato per aggiungere entità alla scena restituisce ora un puntatore all'entità appena inserita, utile per conservare nel main un riferimento agli oggetti complessi al fine di manipolarli.

Per testare la canna da pesca, nel main viene modificato regolarmente il suo stato.



Tappa 6

Dopo la canna da pesca è stata implementata una seconda entità animata, un pesce che, quando viene effettuata una cattura, compare in primo piano e ruota su sé stesso.

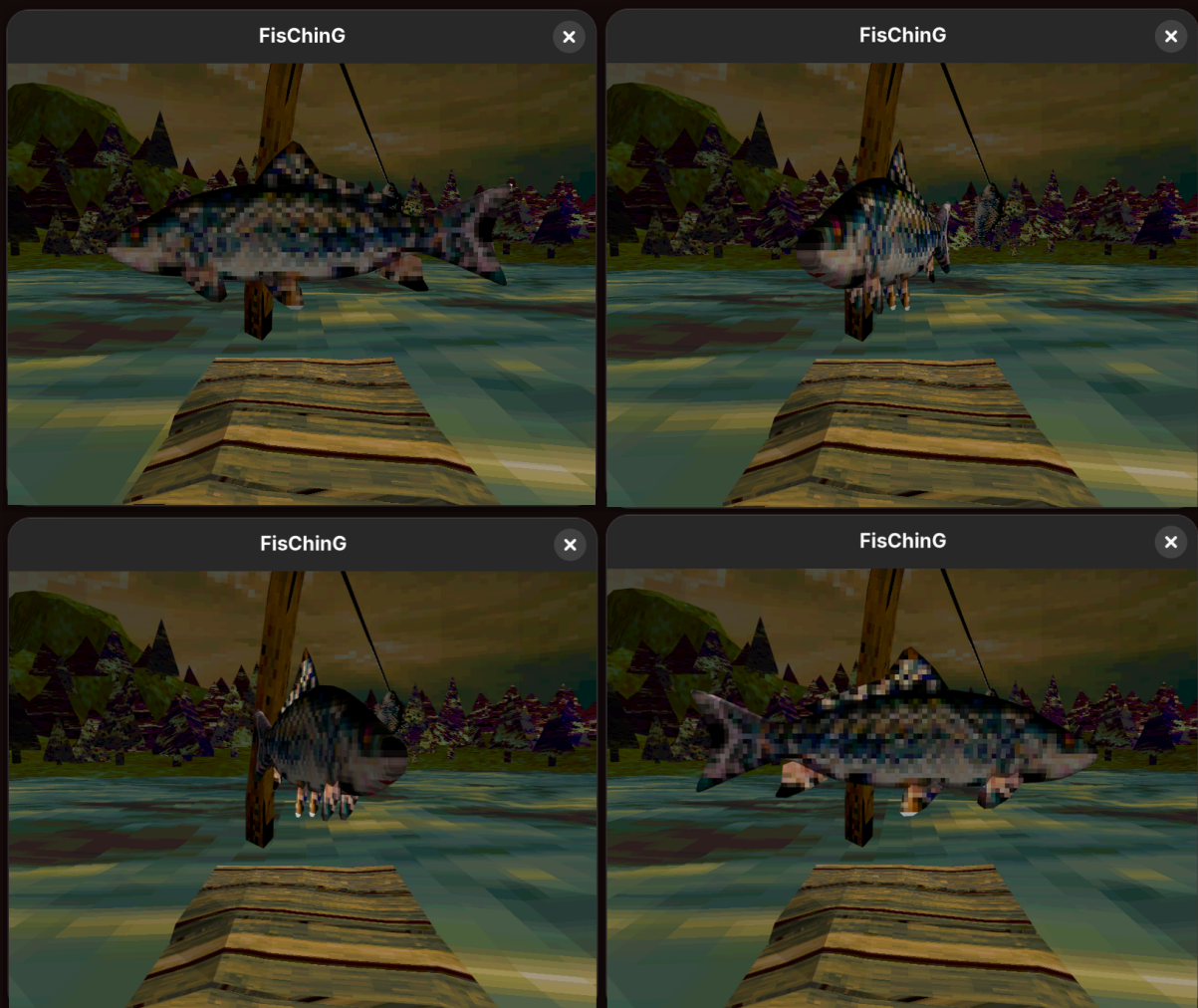
Le classi Entity e Rod, ormai complete, sono state spostate in un file a parte per accorciare il file principale.

È nata la classe Fish, altra derivata di Entity, per implementare i movimenti del pesce. Il modello deve comparire solo in alcuni momenti, la nuova classe contiene quindi un bool, visible, il cui valore indica al metodo draw se deve disegnarlo oppure no.

Come per Rod, il metodo update usa diversi parametri della classe per trasformare il pesce, se visibile.

Dato che il nuovo modello, posizionato tra la canna da pesca e lo spettatore, girando occupava parecchio spazio, interferiva o con la canna, o con la visuale. Si è dovuto quindi introdurre un movimento di camera, in cui lo spettatore si sposta indietro mentre la canna si alza, così che, quando il modello del pesce compare, non ci siano problemi.

Alla classe Camera sono stati quindi aggiunti uno stato CameraState e un metodo update.



Tappa 7

Per separare main e logica di gioco è stata creata la classe Game. Questa contiene tutti gli elementi grafici del gioco (shader, camera e scena), le entità manipolabili (canna da pesca e pesce), un clock e un proprio stato, GameState.

Game si occupa di scandire il tempo per generare eventi interni (passaggi di stato temporizzati) e gestire eventi esterni (clic del mouse), in particolare dopo una quantità casuale di secondi d'attesa (stato WAITING) un pesce abbocca e la canna da pesca vibra (stato REELING), quando viene cliccato un qualsiasi punto dello schermo il pesce viene pescato, innescando il movimento di camera e canna all'indietro e la comparsa del pesce (stati CATCHING e CAUGHT), con un altro clic il giocatore può proseguire, la canna e la camera si resettano (stato RESET) e il gioco ritorna allo stato d'attesa.

La funzione loadScene è diventata un metodo della classe Game, e tutte le altre classi sono state spostate in file dedicati.

Il main ora non fa altro che creare un oggetto Game, aggiornarlo ad ogni ciclo chiamando il suo metodo update, e compiere le tre operazioni principali della grafica 3D, clear-draw (chiamando il metodo render di Game)-display.

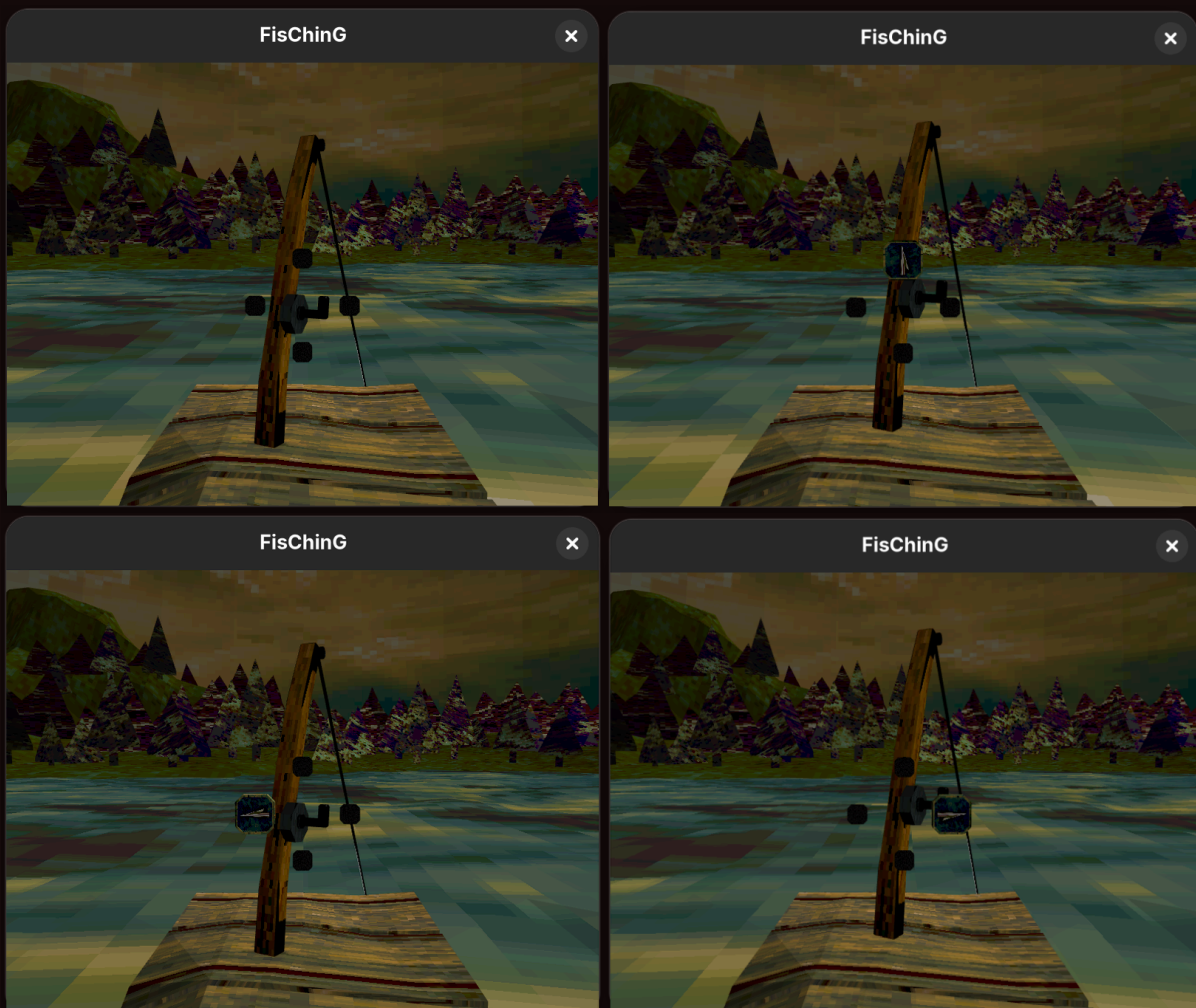
Tappa 8

In questa fase è stato implementato il minigioco vero e proprio. La meccanica consiste nel premere il tasto giusto, mostrato a schermo, entro un tempo limite per un numero fissato di volte. Sono quindi state scritte le classi Button, derivata di Entity, e QTE, da Quick-Time Event.

Button rappresenta i bottoni dell'interfaccia. Se ne istanziano 4 neri, visibili per tutto il minigioco, e 4 colorati, rappresentanti le 4 frecce direzionali; di questi ne è visibile uno per volta, solo quando il tasto corrispondente è da premere.

QTE gestisce invece la logica e il ritmo del gioco, memorizza puntatori alle entità bottoni per gestirne la visibilità, sceglie randomicamente quale tasto va premuto e dopo quanto tempo, e decreta il successo o fallimento del giocatore in base agli eventi da lui generati.

La classe Game contiene un'istanza di QTE, quando un pesce abbocca segnala la partenza del minigioco, poi controlla regolarmente lo stato dell'oggetto, in caso di fallimento resetta lo stato del gioco a quello d'attesa, in caso di successo, invece, mostra il pesce catturato.

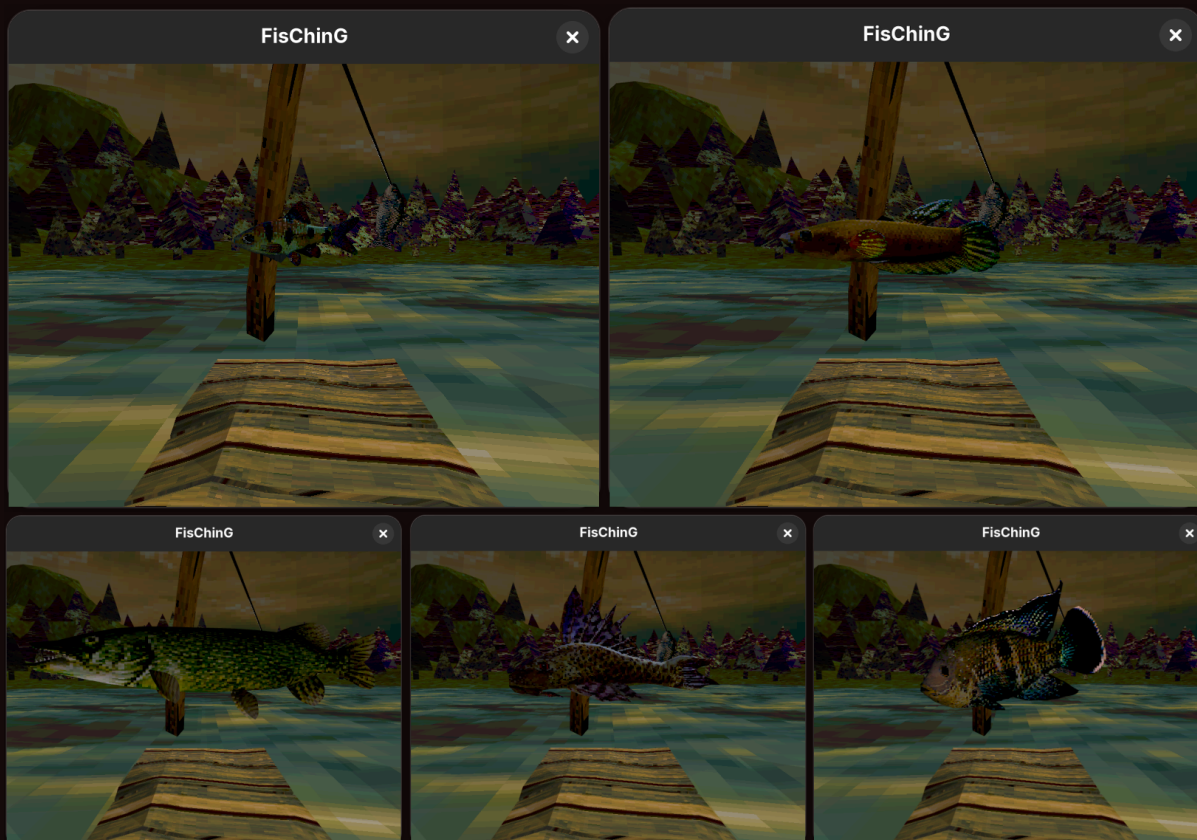


Tappa 9

Per dare varietà al gioco sono stati aggiunti pesci diversi, la cattura di ognuno ha un livello di difficoltà proprio, direttamente proporzionale alla quantità di tasti da premere durante il minigioco.

La classe Game contiene ora un vector di puntatori a Fish, contenente tutti i possibili pesci, e un puntatore al pesce corrente, scelto randomicamente a ogni loop di gioco; i pesci più facili vengono estratti più spesso, mentre quelli più difficili sono anche i più rari.

Game comunica a QTE la difficoltà del pesce corrente quando avvia il minigioco.



Tappa 10

In questa tappa sono stati aggiunti una schermata di avvio del gioco e un punteggio. La finestra è diventata una `RenderWindow` per poter disegnare anche con SFML.

È stato aggiunto un nuovo stato al gioco (stato `START`) durante il quale viene mostrata a schermo una breve introduzione.

Durante gli altri stati, invece, vengono annotati in alto a sinistra i punti accumulati fino a quel momento, calcolati sommando i livelli di difficoltà di tutti i pesci catturati con successo.

Per evitare deformazioni del testo durante il ridimensionamento della finestra viene ora aggiornata anche la vista della finestra.



Tappa 11

Per concludere il progetto sono stati aggiunti alcuni suoni: un sottofondo ambientale, il suono della canna che si agita nell'acqua e il rumore della canna che viene sollevata fuori dall'acqua, riciclato anche per quando torna dentro.

Per farlo è stata usata la classe `Music` di SFML, con la quale sono stati importati dei file mp3 e regolati alcuni loro parametri (ciclicità e volume).

La riproduzione dei suoni è gestita dalla classe `Game`, che mantiene il rumore di fondo sempre in esecuzione mentre gli altri suoni sono fatti partire e fermati quando serve.

Problemi e Soluzioni

La difficoltà più grande riscontrata è stata nella progettazione della struttura del programma, in particolare, per quanto riguarda le classi, la gestione delle dipendenze e la divisione delle responsabilità.

Inoltre, mantenere tutte le classi in un unico file è stato impossibile, perché la quantità di righe presenti ha reso presto difficile ambientarsi nel codice, è stato quindi impiegato molto tempo per studiare un refactoring che fosse semplice ma efficace e ordinato.

La decisione finale è stata, per ogni tappa, di spostare in file .hpp separati le classi dallo sviluppo ormai concluso, così da usare i file principali per concentrarsi sulle nuove funzionalità da implementare.

Conclusioni

Il programma finale soddisfa tutti gli obiettivi che erano stati prefissati. Come previsto, lo sviluppo del codice è stato un ottimo modo per comprendere meglio e approfondire le nozioni di computer grafica viste a lezione, sia teoriche che pratiche.

Crediti

Più volte sono state interrogate delle LLM (Gemini e ChatGPT), principalmente per consigli sulla struttura del progetto e spiegazioni complete e immediate su funzioni delle librerie grafiche. Tutte le risposte ricevute sono state analizzate, e sono state prese in considerazione solo quelle ritenute sensate, facendo un uso responsabile di queste tecnologie.

Per quanto riguarda l'origine delle risorse:

la mesh del pesce base è stata scaricata gratuitamente da internet;

le mesh ambientali (terra, lago e skybox) sono state scolpite da me con Blender;

le restanti mesh e tutte le texture sono state realizzate da Ambra Farinetti, mia amica con esperienza in disegno digitale e modellazione 3D, che ringrazio profondamente per la consulenza;

i file audio sono stati estrapolati da video di Youtube descritti come copyright free.

Un ringraziamento speciale va a Budino, simpatico gatto che ha voluto lasciare un prezioso commento all'interno della tappa 6 (riga 176), grazie Budino.

